



CMPT 295

Unit – Code Optimization

Lecture 35 – Examples of Code Optimization - 2

Last Lecture

- If fast performance is required, s/w developers can optimize programs (make them more time efficient) by exploring various program optimization strategies

- Compiler-generated **optimization strategies**:

1. Reduction in Strength
2. Removing unnecessary function call
3. Removing unnecessary function arguments
4. Code Motion
5. *Inlining* (inline substitution)

argument
elision

- When compiler can perform optimization (not blocked)

- Compiler optimization blockers:

1. Function calls
2. Memory aliasing

Today's Menu

- ▶ Code optimization
 - ▶ Examples of code optimization
 - ▶ Done by compilers
 - ▶ Done by software developers
 - ▶ More optimization strategies - Based on microprocessor architecture
 - ▶ Loop unrolling
 - ▶ Instruction parallelism
 - ▶ Re-association
 - ▶ Accumulators

s/w developer-generated optimization

1. Reduction in strength
2. Code and function calls motion:
 - When they are executed in loops yet producing same result at each iteration -> move them outside the loop
3. Memory references in loops
 - Use a local variable “accumulator” instead -> likely to be held in register
4. Branching minimization (Lab 12 – Part 3)
 - This reduces possibility of mis-prediction by microprocessor and associated penalty
5. *Inlining*
 - Replace function calls with the function implementation (body)

```
// Figures listed in comments below are for  
// 1) no opt. 2) -O1 3) -O2
```

Demo – str_alnum(...) – versions 1 & 2

Code and
function call
motion

```
// Around 72770/16580/3101 clock cycles  
int str_alnum1(char *s) {  
    int i;  
    for (i = 0; i < strlen(s); i++) {  
        if (!isalnum(s[i])) {  
            return 0;  
        }  
    }  
    return 1;  
}
```

```
// Around 23239/10301/1128 clock cycles  
int str_alnum2(char *s) {  
    int i;  
    int len = strlen(s);  
    for (i = 0; i < len; i++) {  
        if (!isalnum(s[i])) {  
            return 0;  
        }  
    }  
    return 1;  
}
```

Inefficiency in str_alnum1(...): calling strlen(s) at every iteration of the for loop

Solution: Because strlen(s) produces the same result at every iteration of the for loop, we can move strlen(s) outside (before) the for loop and assigned its constant result to a local variable, which is used inside the loop

Effect of this solution: Since strlen(s) has a time efficiency of $O(n)$, it made str_alnum1(...) operate in $O(n^2)$. Now that strlen(s) is outside the loop, str_alnum1(...) operates in $O(n)$.

Possible inefficiency in str_alnum2(...): isalnum(s[i]) called inside the loop

Demo – str_alnum(...) - version 3

Inline substitution

```
// Around 54754/17844/4048 clock cycles
int str_alnum3(char *s) {
    int i;
    int len = strlen(s);
    for (i = 0; i < len; i++) {
        if (!(
            ((s[i] >= 'a') && (s[i] <= 'z')) ||
            ((s[i] >= 'A') && (s[i] <= 'Z')) ||
            ((s[i] >= '0') && (s[i] <= '9')))) {
            return 0;
        }
    }
    return 1;
}
```

Solution: We could “inline” `isalnum(s[i])` and save on the function call overhead

Since `isalnum(...)` is a C function, we “inline” this function with our own implementation of it

Why is this solution not working?
Possible reasons:

1. When “inlining” the function `isalnum(...)`, we have introduced 6 `CMP` instructions (i.e., conditional jumps or branches - in assembly code - which can be taken or not). For each of them, the microprocessor can mis-predict, causing instructions smashing, which increases execution time
2. We also have introduced 6 memory accesses

Demo – str_alnum(...) – version 4

Memory
reference
reduction

```
// Around 61124/17199/3905 clock cycles
int str_alnum4(char *s) {
    int i;
    int len = strlen(s);
    char aChar;
    for (i = 0; i < len; i++) {
        aChar = s[i];
        if (!(
            ((aChar >= 'a') && (aChar <= 'z')) ||
            ((aChar >= 'A') && (aChar <= 'Z')) ||
            ((aChar >= '0') && (aChar <= '9')))) {
            return 0;
        }
    }
    return 1;
}
```

Solution: We access memory once and assign its result to a local variable

But: We are still using 6 CMP instructions

Demo – str_alnum(...) – version 5

```
// Around 15400/6574/1059 clock cycles
int str_alnum5(char *s) {
    int i;
    int len = strlen(s);
    for (i = 0; i < len; i++) {
        if (!lookUpTable[s[i]]) {
            return 0;
        }
    }
    return 1;
}
```

4. Branching minimization

```
// Hash Table
// using ASCII code as hash code
```

```
char lookUpTable[128] = {
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1,
    1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0,
    0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
    1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
    1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
    1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1,
    1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
    1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
    1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
```

Solution: We use a “look up” table. Each character of the string **s** is used as an index into this table. The table contains **1** at **s[i]** if this character is either a letter (alpha) or a number (numeric) or **0** otherwise.

Also, the whole look up table may be cached in the fastest section of the memory -> Chapter 6

Summary

- s/w developer-generated optimization:
 - Compiler-generated **optimization strategies**
 - Memory reference (access) reduction
 - Branching minimization
- Also, s/w developers can help the compiler optimizing by transforming programs in such a way that the *optimization blockers* (function calls and memory aliasing) are removed

Next Lecture

- Code optimization
 - Examples of code optimization
 - Done by compilers
 - Done by software developers
 - More optimization strategies - Based on microprocessor architecture
 - Loop unrolling
 - Instruction parallelism
 - Re-association
 - Accumulators